



Instituto Tecnológico de Mazatlán

Comunicación I2C entre 4 Arduinos

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Hernández Barrón

Alumnos:

Rosa Guadalupe Romero Navarro

Javier Ramón Quintero Valdez

Gabriel Alejandro Barrón Sarabia

Reporte de práctica: Comunicación I2C entre 4 Arduinos

1. Descripción del proyecto

Esta práctica consiste en implementar y comprender el funcionamiento del bus de comunicación I2C mediante el enlace de un Arduino maestro con tres Arduinos esclavos, todos compartiendo únicamente las líneas SDA y SCL más una tierra común, y diferenciándose entre sí por una dirección propia (0x08, 0x09 y 0x0A).

El maestro se encarga de coordinar toda la comunicación: envía órdenes al esclavo que controla un LED, envía ángulos al esclavo que mueve un servomotor, y solicita periódicamente una lectura de potenciómetro al tercer esclavo. Con esta información el maestro convierte el valor leído en un ángulo de 0° a 180° y lo reenvía al servomotor, mientras informa por el Monitor serie el estado de la comunicación y si algún esclavo no responde.

2. Objetivos

- Comprender el funcionamiento del bus I2C y su esquema maestro-esclavo.
- Armar en Tinkercad un bus I2C compartido entre cuatro Arduino Uno.
- Asignar direcciones únicas a cada esclavo (0x08, 0x09 y 0x0A).
- Programar el envío de órdenes y datos desde el maestro hacia los esclavos.
- Programar la recepción de datos en los esclavos mediante `Wire.onReceive()` y `Wire.onRequest()`.
- Detectar fallas de comunicación mediante el valor de retorno de `endTransmission()`.
- Utilizar `millis()` en el maestro para no bloquear la atención al Monitor serie.

3. Herramientas y material utilizado

Hardware (simulado en Tinkercad):

- 4 Arduino Uno R4 WiFi
- 1 protoboard (Breadboard Small)
- 1 LED
- 1 resistencia de 470 Ω (LED) y 2 resistencias de 4.7 k Ω (pull-up en SDA y SCL)
- 1 micro servomotor
- 1 potenciómetro
- Cables de conexión

Software:

- Librerías Wire y Servo (incluidas en el entorno de Arduino)
- Monitor serie

4. Diagrama

El diagrama muestra la conexión de los cuatro Arduino compartiendo las líneas SDA (A4) y SCL (A5), así como la tierra común. El primer Arduino controla el LED, el segundo el servomotor, el tercero lee el potenciómetro, y el cuarto actúa como maestro y se conecta a la computadora por USB.

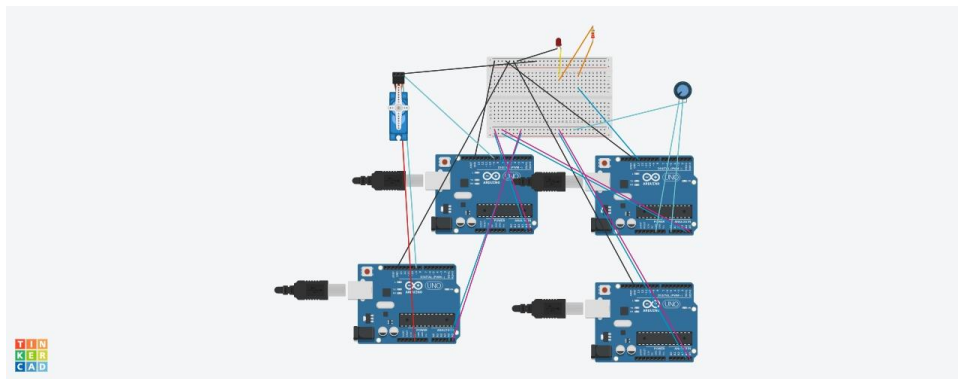
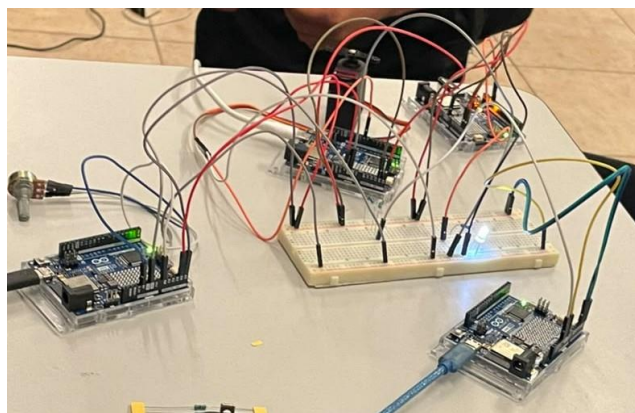


Diagrama del bus I2C armado en Tinkercad.

5. Montaje físico

Como parte de la práctica se realizó también el armado físico del circuito en protoboard, replicando las conexiones simuladas en Tinkercad y verificando de forma presencial el comportamiento del bus I2C.



Montaje físico del circuito.

6. Metodología

Primero se definieron las direcciones de cada esclavo (0x08 para el LED, 0x09 para el servomotor y 0x0A para el potenciómetro) y se conectaron las líneas SDA, SCL y GND de los cuatro Arduino en un mismo bus.

En cada esclavo se inicializó la librería Wire con `Wire.begin(direccion)` y se registró la función correspondiente: `onReceive()` para los esclavos que reciben datos (LED y servomotor) y `onRequest()` para el esclavo que responde con la lectura del potenciómetro.

En el maestro se utilizó `Wire.begin()` sin dirección, y dentro del `loop()` se combinaron dos tareas sin bloquear la ejecución: la lectura del Monitor serie para encender o apagar el LED, y una consulta periódica al potenciómetro cada 500 ms usando `millis()`, cuyo resultado se convierte en un ángulo con `map()` y se envía al servomotor.

Después de cada `beginTransaction()/endTransmission()` y de cada `requestFrom()`, el maestro comprueba el resultado para detectar si el esclavo correspondiente no respondió, informando la falla por el Monitor serie en vez de detener el programa.

7. Explicación del funcionamiento

El maestro es quien controla toda la comunicación: decide con qué esclavo habla, en qué momento y si le envía una orden o le solicita un dato; el esclavo únicamente responde cuando es llamado por su dirección.

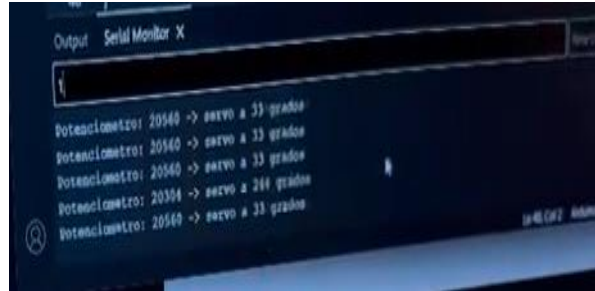
Para el LED, el maestro envía un solo byte (1 o 0) que el esclavo interpreta con `digitalWrite()` dentro de su rutina de recepción. Para el servomotor, el maestro envía un byte con el ángulo calculado, y el esclavo lo guarda en una variable para moverlo fuera de la rutina de interrupción, evitando bloquear la comunicación I2C mientras se mueve el servo.

Para el potenciómetro, el esclavo responde a la petición del maestro enviando el valor de `analogRead()` dividido en dos bytes (`highByte` y `lowByte`), ya que el valor puede llegar hasta 1023 y no cabe en un solo byte. El maestro reconstruye el valor original combinando ambos bytes con un desplazamiento de bits.

Un error identificado durante la práctica fue leer el mismo dato de I2C más de una vez dentro de una misma expresión: como `Wire.read()` consume el byte en la primera lectura, las lecturas siguientes devuelven un valor inválido, por lo que cada dato debe leerse una sola vez y guardarse en una variable.

8. Monitor Serie

Con el bus ya funcionando, el Monitor serie del maestro muestra en tiempo real el valor leído del potenciómetro, el ángulo calculado para el servomotor, y los mensajes de aviso cuando algún esclavo no responde. Además, al escribir 1 o 0 en el Monitor serie se confirma el cambio de estado del LED.



Monitor Serie mostrando las lecturas del potenciómetro, el ángulo enviado al servomotor

9. Análisis de resultados

Durante las pruebas se observó que los tres esclavos respondieron correctamente a sus respectivas direcciones sin interferir entre sí, lo que confirmó que basta con una dirección distinta para diferenciarlos en un mismo bus de solo dos líneas de datos.

El movimiento del servomotor respondió de forma consistente a los cambios del potenciómetro, y el LED cambió de estado inmediatamente después de escribir 1 o 0 en el Monitor serie, incluso mientras el maestro seguía consultando el potenciómetro cada 500 ms gracias al uso de `millis()` en lugar de `delay()`.

También se comprobó la utilidad de revisar el resultado de `endTransmission()` y de `requestFrom()`: al desconectar temporalmente un esclavo, el maestro detectó la falta de respuesta (NACK) y lo reportó por el Monitor serie en lugar de quedarse bloqueado esperando una respuesta que nunca llega.

Por último, se identificó un error de armado frecuente en la protoboard: colocar ambas patas de la resistencia del LED en la misma columna anula su efecto, ya que los huecos de una columna están unidos internamente, lo que permite el paso de corriente sin resistencia y puede dañar el LED; el error se corrigió ubicando cada pata en una columna distinta.

10. Conclusiones

Gabriel

En esta práctica comprendí cómo el bus I2C permite comunicar varios dispositivos utilizando solamente dos líneas de datos y una tierra común, y que agregar un nuevo esclavo no requiere pines adicionales, sino únicamente una dirección distinta. También entendí por qué dos esclavos no pueden compartir la misma dirección, ya que ambos responderían al mismo tiempo y los datos se corromperían.

Javier

Durante la práctica aprendí la importancia de revisar el resultado de `endTransmission()` y de `requestFrom()`, ya que permiten detectar cuando un esclavo no contesta en lugar de que el programa se quede esperando indefinidamente. También reforcé la idea de leer un dato de I2C una sola vez y

guardarlo en una variable, pues leerlo varias veces dentro de la misma expresión devuelve valores inválidos.

Rosa

Esta práctica me permitió comprender cómo un solo maestro puede coordinar a varios esclavos con tareas distintas (encender un LED, mover un servomotor y leer un potenciómetro) sin bloquear el programa, gracias al uso de `millis()` en lugar de `delay()`. Además, aprendí a identificar errores comunes de armado en la protoboard, como una resistencia mal ubicada, que pueden dañar los componentes si no se revisan con cuidado.